

Code Hardener

ATTESTATION CERTIFICATE



bulletproof-context-guard

Scan 7cbe07b4 | Branch: main | Jul 24, 2026

Attestation ID

60af1c4a-f8ae-457d-bd81-...

Type

in-toto

Subject

bulletproof-context-guard

Created

Jul 24, 2026, 3:25 PM

Subject Digest

sha256:5bfc1ac6ab2c63e3ef30e4c409fee56b4033c1ed4936d59cc07cb90f...

Signature Status

CRYPTOGRAPHICALLY SIGNED

Algorithm: Ed25519 (Local Signing Key)

Signature

0lNfggk4tqMrmfyor2Isk6ZKtPu+0RSiNTL4ahoPIFN5lK5SyKY8B9WJ42SB+Pri
aS/aWKj31HFcV3Pi28KdAQ==

This attestation certifies the security scan was performed and results were recorded with cryptographic integrity.

Scan Report — bulletproof-context-guard

Branch: main | Status: completed | Jul 24, 2026

946 / 1000 Score — Quality: excellent

Scan Details

Profile: standard
Duration: 12s
Started: Jul 24, 2026, 3:25 PM
Completed: Jul 24, 2026, 3:25 PM
Files Analyzed: 21
Scanners Run: 10 of 12
Open Findings: 9
Resolved / Suppressed:0
Total Detected: 9
Codebase: Shell: 8 files | Config: 6 files | Python: 3 files | Markdown: 1 files

0 Critical 0 High 7 Medium 2 Low 0 Info

Scanner Results

12 tools executed — 7 passed | 3 with open findings | 2 not applicable | 0 errors

Scanner	Category	Outcome	Findings	Duration	Files Scanned
trivy	SCA	FAIL	1	1s	1
gitleaks	Secrets	PASS	0	0s	21
opengrep	SAST	PASS	0	5s	11
checkov	IaC	PASS	0	2s	6
grype	SCA	PASS	0	1s	21
syft	SBOM	FAIL	1	1s	1
package-validator	AI Code Quality	N/A	0	0s	21

oxlint	Code Quality	N/A	0	0s	11
ruff	Code Quality	FAIL	7	0s	3
actionlint	CI/CD Security	PASS	0	0s	6
jscpd	Code Quality	PASS	0	0s	3
typos	Code Quality	PASS	0	0s	21

Scanner Evidence & Audit Trail

trivy (SCA)

FINDINGS

Scan Scope: Scanned 1 target(s): filesystem

Files Analyzed: 1

Rules Checked: 4

Checks Performed:

- Vulnerability scanning
- Misconfiguration detection
- Secret detection
- License compliance

Standards: CVE/NVD, OWASP Top 10 (A06), CIS Benchmarks, NIST SP 800-53

Methodology:

Scans filesystem for package manifests and lock files, matches installed packages against vulnerability databases (NVD, GitHub Advisory, vendor-specific). Also checks for misconfigurations and secrets using built-in rules.

Targets Analyzed:

Loose File License(s)

gitleaks (Secrets)

PASS

Scan Scope: Working directory scanned for 150+ secret patterns with entropy analysis

Configuration: Custom gitleaks.toml

Files Analyzed: 21

Rules Checked: 150

Checks Performed:

- AWS credential detection
- API key detection
- Private key detection
- Generic secret patterns
- High-entropy string analysis
- Token detection

Standards: CWE-798, CWE-259, OWASP Top 10 (A07)

Methodology:

Regex-based pattern matching against 150+ secret patterns with entropy analysis. Scans current file contents and optionally git commit history. Applies allowlists and custom rules to reduce false positives. Secrets are redacted in output.

opengrep (SAST)

PASS

Scan Scope: Multi-language SAST with auto, security-audit, owasp-top-ten, cwe-top-25, r2c-security-audit, and secrets rulesets

Configuration: Custom semgrep-rules.yml + default rulesets

Files Analyzed: 11

Checks Performed:

- OWASP Top 10 patterns
- Security audit rules

- Secrets detection
- Injection patterns
- XSS patterns
- Insecure crypto usage

Standards: OWASP Top 10, CWE Top 25, SANS Top 25

Methodology:

Pattern-based static analysis using Semgrep rule engine with auto, security-audit, owasp-top-ten, and secrets rulesets. Matches AST patterns against known vulnerability signatures.

checkov (IaC)

PASS

Scan Scope: IaC security analysis — 0 files, 0 checks (0 passed, 0 failed), frameworks: auto-detected

Configuration: Custom .checkov.yaml

Files Analyzed: 6

Checks Performed:

- Terraform misconfigurations
- CloudFormation template issues
- Kubernetes manifest hardening
- Dockerfile best practices
- Helm chart security
- GitHub Actions security
- GitLab CI security
- Secrets in IaC files

Standards: CIS Benchmarks, NIST SP 800-53, SOC 2, PCI-DSS, HIPAA

Methodology:

Graph-based static analysis of infrastructure-as-code files. Parses IaC definitions, builds resource graphs, and checks 1000+ built-in policies covering security, networking, encryption, and compliance.

grype (SCA)

PASS

Scan Scope: SCA analysis: 0 unique CVEs across 0 vulnerable packages (filesystem)

Configuration: Grype with --by-cve deduplication and --add-cpes-if-none

Files Analyzed: 21

Checks Performed:

- CVE vulnerability matching
- Dependency version analysis
- CPE-based matching
- Advisory cross-referencing
- Fix availability detection

Standards: CVE/NVD, OWASP Top 10 (A06), GHSA

Methodology:

Generates SBOM using Syft, then matches all identified packages against vulnerability databases (NVD, GitHub Advisories, OS vendor databases). Reports CVEs with severity, fix versions, and references.

syft (SBOM)

FINDINGS

Scan Scope: SBOM of 1 packages across 0 language(s): N/A

Configuration: Syft with JSON + CycloneDX output for SBOM standards compliance

Files Analyzed: 1

Rules Checked: 4

Checks Performed:

- SBOM generation (Syft JSON + CycloneDX)
- License compliance analysis
- Copyleft license detection (AGPL, GPL, MPL)
- Weak copyleft classification (LGPL)
- Unknown license flagging

- Package URL (PURL) generation
- CPE generation

Standards: NTIA SBOM Minimum Elements, CycloneDX, SPDX, Executive Order 14028

Methodology:

Catalogs software packages by parsing package manifests, lock files, and installed package databases. Generates standardized SBOM in CycloneDX or SPDX format for supply chain visibility.

package-validator (AI Code Quality)

N/A

Scan Scope: Dependency files (package.json, requirements.txt, go.mod) validated against package registries

Files Analyzed: 21

Checks Performed:

- npm registry existence validation
- PyPI registry existence validation
- Go module proxy existence validation
- Slopsquatting attack vector detection
- Hallucinated dependency identification

Standards: CWE-829, OWASP A08:2021, SLSA Supply Chain Security

Methodology:

Parses dependency manifests and validates each package exists in its respective registry (npm, PyPI, Go proxy) via HTTP HEAD requests. Non-existent packages are flagged as hallucinated — a common AI code generation artifact that creates supply chain attack vectors.

oxlint (Code Quality)

N/A

Scan Scope: JavaScript (.js, .jsx) and TypeScript (.ts, .tsx) source files

Files Analyzed: 11

Checks Performed:

- Code correctness checks
- Suspicious code detection
- Performance anti-patterns
- Import/export validation

Standards: CWE, ESLint Core Rules

Methodology:

Rust-based linter that performs AST analysis of JS/TS files at high speed. Detects correctness issues, suspicious patterns, and performance anti-patterns.

ruff (Code Quality)

FINDINGS

Scan Scope: Python linting and security analysis via Ruff

Configuration: Default Ruff rules

Files Analyzed: 3

Checks Performed:

- Pyflakes error detection
- pycodestyle enforcement
- flake8-bandit security checks
- isort import ordering
- pep8-naming conventions
- pyupgrade modernization
- flake8-bugbear bug detection
- flake8-comprehensions optimization

Standards: PEP 8, PEP 257, CWE, OWASP Top 10

Methodology:

Rust-based Python linter implementing 800+ rules from flake8, pylint, isort, and others. 10-100x faster than traditional Python linters.

actionlint (CI/CD Security)

PASS

Scan Scope: .github/workflows/*.yaml files

Files Analyzed: 6

Checks Performed:

- Workflow syntax validation
- Action version pinning checks
- Shell script validation
- Expression type checking
- Permissions analysis

Standards: GitHub Actions Security Best Practices, CWE-78

Methodology:

Static analysis of GitHub Actions workflow files. Validates syntax, checks for unpinned action versions, analyzes shell scripts for errors, and verifies expression types.

jscpd (Code Quality)

PASS

Scan Scope: All source files across supported languages

Files Analyzed: 3

Checks Performed:

- Cross-file duplicate detection
- Code clone identification
- Duplication percentage calculation

Standards: ISO/IEC 25010 (Maintainability)

Methodology:

Token-based analysis to detect copy-pasted code blocks across files and languages. Reports duplication percentage and identifies specific cloned regions.

typo5 (Code Quality)

PASS

Scan Scope: All text files in the project

Files Analyzed: 21

Checks Performed:

- Identifier spell checking
- String literal spell checking
- Comment spell checking
- Filename spell checking

Standards: ISO/IEC 25010 (Maintainability)

Methodology:

Fast, low-false-positive spell checker designed for source code. Checks identifiers, strings, and comments against a curated dictionary.

Open Findings (9)

ruff

Code Quality | 7 findings | Ultra-fast Python linter (pyflakes, pycodestyle, flake8-bandit)

Severity	Title	File	CWE / Rule
MEDIUM	Ruff F841: Local variable `current_output` is assigned but never used	mcp/server.py:110	RUFF-F841
Fix:	Auto-fix available: Remove assignment to unused variable `current_output` (applicability: unsafe)		

MEDIUM	Ruff F841: Local variable `current_input` is assigned... Local variable `current_input` is assigned to but never used	mcp/server.py:109	RUFF-F841
Fix:	Auto-fix available: Remove assignment to unused variable `current_input` (applicability: unsafe)		
MEDIUM	Ruff F841: Local variable `cache_read` is assigned to... Local variable `cache_read` is assigned to but never used	mcp/server.py:108	RUFF-F841
Fix:	Auto-fix available: Remove assignment to unused variable `cache_read` (applicability: unsafe)		
MEDIUM	Ruff F841: Local variable `cache_creation` is assigne... Local variable `cache_creation` is assigned to but never used	mcp/server.py:107	RUFF-F841
Fix:	Auto-fix available: Remove assignment to unused variable `cache_creation` (applicability: unsafe)		
MEDIUM	Ruff F841: Local variable `remaining_pct` is assigned... Local variable `remaining_pct` is assigned to but never used	mcp/server.py:104	RUFF-F841
Fix:	Auto-fix available: Remove assignment to unused variable `remaining_pct` (applicability: unsafe)		
MEDIUM	Ruff F401: `time` imported but unused `time` imported but unused	mcp/server.py:21	RUFF-F401
Fix:	Auto-fix available: Remove unused import: `time` (applicability: safe)		
MEDIUM	Ruff F401: `os` imported but unused `os` imported but unused	mcp/server.py:19	RUFF-F401
Fix:	Auto-fix available: Remove unused import: `os` (applicability: safe)		

syft

SBOM | 1 finding | Software bill of materials generator for supply chain visibility

Severity	Title	File	CWE / Rule
LOW	Unknown License: actions/checkout@11d5960a326750d5838... Package actions/checkout has no license information. This may indicate a compliance risk.	/.github/workflows/ci.yml	SBOM-LICENSE-UNKNOWN
Package: actions/checkout Version: 11d5960a326750d5838078e36cf38b85af677262 Type: github-action Fix: Verify the license for this package before using in production. OWASP: A08:2021-Software and Data Integrity Failures			

trivy

SCA | 1 finding | Container image and filesystem vulnerability scanner

Severity	Title	File	CWE / Rule
LOW	License Compliance: Apache-2.0 in Package uses license Apache-2.0 (category: notice). Review for compliance with your organization's license policy.	LICENSE	LICENSE-Apache-2.0
Package: License: Apache-2.0 Category: notice Fix: Review this finding and apply the appropriate fix based on the description: Package uses license Apache-2.0 (category: notice). Review for compliance with your organization's license policy. OWASP: A06:2021-Vulnerable and Outdated Components			

Codebase Coverage (21 files analyzed)

This scan analyzed 21 files using 10 security tools across 8 testing categories. Each scanner targets files relevant to its analysis domain (e.g., SAST scanners analyze source code, SCA scanners analyze dependency manifests).

Files by Language:

Shell: 8 files	Config: 6 files
Python: 3 files	Markdown: 1 files

Python: 3 files Markdown: 1 files

Files by Extension:

```
.sh: 8                                .json: 5                                .py: 3
.other: 2                             .yaml: 1                                .gitignore: 1
```

```
.other: 2
.md: 1
.yml: 1
.gitignore: 1
```

Per-Scanner File Coverage:

Scanner	Category	Files Analyzed	Scope
trivy	SCA	1	Scanned 1 target(s): filesystem
gitleaks	Secrets	21	Working directory scanned for 150+ secret p...
opengrep	SAST	11	Multi-language SAST with auto, security-aud...
checkov	IaC	6	IaC security analysis — 0 files, 0 checks (...)
grype	SCA	21	SCA analysis: 0 unique CVEs across 0 vulner...
syft	SBOM	1	SBOM of 1 packages across 0 language(s): N/A
package-validator	AI Code Quality	21	Dependency files (package.json, requirement...
oxlint	Code Quality	11	JavaScript (.js, .jsx) and TypeScript (.ts,...)
ruff	Code Quality	3	Python linting and security analysis via Ruff
actionlint	CI/CD Security	6	.github/workflows/*.yaml files
jscpd	Code Quality	3	All source files across supported languages
typos	Code Quality	21	All text files in the project

Files with Findings (3):

File Path	Findings	Scanners
mcp/server.py	7	ruff
./github/workflows/ci.yml	1	syft
LICENSE	1	trivy

Software Bill of Materials (1 packages)

Package types: github-action: 1

Top licenses: Unknown (1)

Package	Version	Type	Language	License
actions/checkout	11d5960a326750	github-actio	-	Unknown